

A Hacker's Guide to the Logic Theorist

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved. (See copyright details at the end of this document.)

For the curious visitor who wants to read the 1963 IPL-V source of the first AI program — the Logic Theory Machine as it was rebuilt to be read

////////////////////////////////////

Before You Arrive: What Is the Logic Theorist?

The Logic Theorist (LT) was created by Allen Newell, J. C. (Cliff) Shaw, and Herbert Simon at the RAND Corporation and Carnegie Tech in 1955–1956. It discovered proofs of theorems in the propositional calculus of Whitehead and Russell's *Principia Mathematica* by heuristic search, and it is widely regarded as the first artificial intelligence program. In January 1956 Simon walked into a classroom and announced, "Over Christmas Allen Newell and I invented a thinking machine." By 1957 the machine version had proved 38 of the first 52 theorems of *Principia* Chapter 2 — and for Theorem *2.85 it found a proof shorter and more direct than Whitehead and Russell's own. Simon sent it to Bertrand Russell, who responded with delight.

A note on what you're reading. The program in this directory is *not* the 1956 original. LT went through as many lives as its host language, IPL:

- **LT1 (1956)** was written in IPL-I — a "pseudocode" for an abstract machine that was never implemented on any computer. It was executed *by hand*, famously using Simon's wife, children, and graduate students as the CPU. About 400 lines.
- **LT2 (1956–57)**, in IPL-II, ran on RAND's JOHNNIAC (4,096 forty-bit words) and produced the first machine proof in August 1956.
- **LT5 (1963)** — the code you are visiting — is the IPL-V version. Fred Tonge converted LT to IPL-V, and Einar Stefferud then rebuilt it as a *pedagogical model* for teaching IPL programming, documenting every routine in RAND Memorandum RM-3731, "The Logic Theory Machine: A Model Heuristic Program." Stefferud simplified code that was "unjustifiably hard to explain," added a new method of replacement on subexpressions, and — his own term-project contribution — an indexed store of true expressions called the *axiom map*. The Memorandum contains the complete listing: nearly 3,000 card images,

every one commented.

So you are reading the first AI the way the first generation of AI students read it: as a worked example, deliberately polished for inspection. The trade is a fair one — what you lose in 1956 authenticity you gain in 1963 legibility.

IPL itself deserves a word. It introduced list processing, symbols as first-class objects, dynamic memory allocation, recursion, and generators (iterators), and it is a direct, acknowledged ancestor of Lisp. IPL-V was the only version released to the public, with implementations on the IBM 650/704/709/7090/7094, Philco 2000, CDC 1604, and more. In 2026 this code was reanimated: Shrager wrote a fresh IPL-V interpreter in Common Lisp and ran the LT listing transcribed directly from Stefferud's report, with more than 99% of the running code unchanged from 1963 (see [arXiv:2603.13514](https://arxiv.org/abs/2603.13514) and github.com/jeffshrager/IPL-V). So unlike most code of its vintage, this program *runs* — you can finish this guide and then watch the artifact prove theorems.

The Map: Nine Districts

[The Charter]	the input deck — axioms, definitions, and the theorems of Principia, as punched cards
[The Substrate]	the IPL-V abstract machine: cells, H1 as both program counter and call stack, J-functions
[City Hall]	executives M1/M2 — select problems, apply methods, manage the subproblem tree
[The Matching Works]	M12 and M110–M115 — substitution and the recursive match, the heart of LT
[The Workshops]	M11, M13–M17 — detachment, replacement, and chaining: the subproblem factories
[The Customs Office]	M40–M43 — utility tests; what counts as a problem worth keeping
[The Card Catalog]	L4 and M54/M62/M63 — the axiom map, a 1963 answer to "too much information"
[The Machine Shops]	the P and Q routines — expression plumbing, with some surprisingly modern tricks
[The Press Office]	M70–M89 — reading expressions in, printing proofs out (and one glorious hack)

Presiding over everything is the run executive X1, with a small Bureau of Debugging (X10–X19) next door. We'll end there.

District 1: The Charter — the input deck

LT's knowledge arrives on punched cards at the end of the deck, after the line

`KICK OFF FOR PROVING THEOREMS`. Three sets of logic expressions follow, separated by blank cards: the true expressions (definitions and axioms), the problems to prove, and a third set we'll get to in a moment.

```
*1.01 ((PIQ).=.(-PVQ)) DEF.
*2.33 ((PVQVR).=.( (PVQ)VR)) DEF.
*3.01 ((P*Q).=-(-(PV-Q)) DEF.
*4.01 ((P=Q).=.( (PIQ)*(QIP))) DEF.
*1.2  ((AVA)IA)
*1.3  (BI(AVB))
*1.4  ((AVB)I(BVA))
*1.5  ((AV(BVC))I(BV(AVC)))
*1.6  ((BIC)I((AVB)I(AVC)))

*2.01 ((PI-P)I-P)
*2.02 (QI(PIQ))
...
```

The notation is constrained by the keypunch: `I` is IMPLIES, `V` is OR, `*` is AND, `-` is NOT, `=` is equivalence, and `.=.` is definitional equivalence (the periods are delimiters that mark `=` for replacement by a different internal symbol). Axioms use the **free variables** A–G; problems use the **bound variables** P–T. The distinction matters: free variables can have things substituted *for* them, bound variables cannot — they mark the parts of an unproved conjecture that must be matched exactly. A theorem earns its free variables only by being proved.

★ DON'T MISS

Definition *2.33 in the standard deck is *broken on purpose*. LT's expressions are strictly binary — every connective takes exactly two subexpressions — so `((PVQVR))` cannot be parsed. Stefferud says so in the text ("Definition *2.33 will fail in conversion") and shipped it anyway, so that every student's first run demonstrates the error handling. The output reads:

```
BAD EXPRESSION ((PVQ/UGH/VR).=.( (PVQ)VR))
```

That `/UGH/` is the external name of `/14`, the "dummy character symbol" the parser splices into a list-form expression at the exact spot where it gave up. The vocabulary listing deadpans: `DUMMY CHARACTER SYMBOL WITH EXTERNAL NAME '/UGH/'`. Error

And the third set of expressions? It sits after the second blank card, where the run executive never reads it. It contains all the remaining theorems of *Principia* *2 through *5 — placed there, Stefferud explains, purely so the Memorandum would document them. The input deck doubles as an appendix.

////////////////////////////////////

District 2: The Substrate — the IPL-V machine

You cannot read LT without reading IPL-V, so here is the machine in one sitting.

IPL-V is an abstract machine built of **cells**, each named by a symbol. A cell holds a P digit, a Q digit, a SYMB, and a LINK; lists are chains of cells connected by LINKs. Everything — data, routines, the program itself — is a list of symbols. A handful of system cells run the show: **H0** is the communication cell (arguments go in, results come out, and it's a push-down stack, so "(0)" and "(1)" in the comments mean the top and second elements of H0); **H1** is the program counter; **H2** is the free-space list; **H3** counts interpretation cycles; **H5** is the test cell, holding + or -. Ten working cells W0–W9 are scratch storage, and every cell is itself a stack — you `preserve` (push) and `restore` (pop) W0 around any use of it, which is how IPL-V gets re-entrant recursion with no other machinery.

An instruction is the same four fields. P is the operation (0 = execute, 1 = input to H0, 2 = output, 3 = restore, 4 = preserve, 5 = replace, 6 = copy, 7 = **branch if H5–**), and Q is the addressing mode (0 = the symbol itself, 1 = the symbol in the cell named, 2 = doubly indirect). So in the listing, `40W0` means "push W0," `11W0` means "input the contents of W0 to H0," `709-4` means "if the last test failed, go to local label 9-4," and `00M43` means "execute routine M43." Labels of the form `9-1`, `9-100` are local symbols, private to the routine that defines them.

The built-in operations are the **J-functions** — about 150 of them, from `J2` (test equal) through list surgery (`J60` locate next, `J65` insert at end, `J74` copy list structure) to the *generator* protocol: `J100` iterates a list, firing a subprocess on each element. Higher-order functions, in 1958.

★ DON'T MISS

H1, the program counter, is also the call stack. To call a routine you push its name onto H1; the interpreter executes whatever H1 names and pops on return. No separate frame mechanism exists or is needed. And the push-down stacks themselves are built out of the same linked cells as everything else. Like the Lisp it fathered, IPL-V is lists all the way down — including the part of the machine that appears to be hardware.

A note on modern equivalents: if you know Forth, the flavor of IPL-V code — tiny stack-shuffling instructions, words built from words — will feel familiar, except that the underlying substance is linked lists rather than a cell array. The generator protocol (J17 set up, J18 fire the subprocess, J19 clean up) is exactly an iterator with a callback. And the H5 test cell discipline — every TEST sets H5, every 7x instruction branches on it — is a one-bit condition code register, which is why the flow diagrams in the Memorandum are all "if H5+ ... if H5-."

District 3: City Hall — M1 and M2

The vocabulary listing gives the street plan: M1–M9 executives, M10–M19 methods, M40–M49 utility measures, M50–M69 information storage and retrieval, M70–M89 input-output, M90–M99 limit testers, M110–M119 match processes. Once you've internalized that numbering, you can navigate the listing without a map.

M2, the multiple-problem executive, generates problems off list L3 and feeds them one at a time to M1. M1's whole strategy fits on one card-page (this is the actual code, lightly trimmed):

M1 SINGLE PROBLEM EXECUTIVE FOR	M1	00M3	
PROBLEM (0). H5 + IF SUCCEEDS		40W0	
		60W0	1W0=PROB
TEST UTILITY.		00M43	
IF NO GOOD, QUIT.		709-4	
PRINT 'TO PROVE' PROBLEM 1W0.		00M78	
TRY SUBSTITUTION.		00M12	
IF WORKED, PRINT PROOF.		70	9-2
	9-1	11W0	
CREATE LIST OF METHODS FOR PROB.		00M8	
APPLY METHODS		00M7	
IF PROOF FOUND, PRINT IT.		70	9-2
TEST IF ANY LIMITS EXCEEDED.		00M90	
IF YES, QUIT.		70	9-3
FIND NEXT SUBPROBLEM.		00M60	
IF NONE, QUIT.		709-3	
PRINT SUBPROBLEM, TRY METHODS.		00M70	9-1
PRINT PROOF FROM (0).	9-2	00M71	
AND QUIT +.		30W0	J4
PRINT FAILURE, QUIT -.	9-3	00M72	9-5

Try substitution (the only method that proves things directly); if it fails, apply the other methods, which manufacture *subproblems* — surrogate expressions whose proof would imply proof of the

original. New subproblems go on the **untried list** (L10); M60 fetches the next one; loop until proof, exhaustion, or a limit trips. The collection of subproblems grows into a tree of hypothesized proof sequences, each node carrying attributes recording which problem it came from, which theorem, and which method (Q10–Q14) — which is exactly the information the proof printer will later walk backward.

Stefferd points out a structural subtlety worth savoring: M1 works *iteratively* while the proof tree grows *recursively*, because derivation context is stored on each subproblem rather than on any stack. This lets the executive jump to whatever part of the tree looks profitable, instead of being trapped in depth-first order. In modern vocabulary: L10 is the open list, L11 (the found problems) is the closed set, and M1 is a best-first search loop — a decade before that vocabulary existed.

The main heuristic, the text explains, is simply **working backward** from the problem toward the axioms. Working forward from true expressions would generate theorems independent of the goal — no better, Stefferud notes, than the "British Museum Algorithm," the era's name for exhaustive enumeration.

★ DON'T MISS

M8 builds each problem's method list by copying L7 — replacement, detachment, forward chaining, backward chaining — and then *tests whether this is an original problem* (J130: does it have a regional name like *2.06, rather than a generated subproblem number?). Only originals get the special list L6 spliced in front: the sublevel replacement methods M16 and M17. The deepest rewriting machinery in LT is reserved for the top of the tree; subproblems get the cheap methods only. Search control by class privilege.

District 4: The Matching Works — M12 and M110–M115

"Matching is the heart of LT," says the Memorandum, and the listing agrees: the match family M110–M117 is where the densest code lives.

The deceptive part first: **M12, the substitution method, does no substitution.** It is an executive. It fetches a list of plausible theorems (from the Card Catalog, District 7), and generates them one at a time against the problem, calling the test-for-match routine M114 — which is *also* an executive, existing only to discard the output of M111 (TEST routines must leave nothing in H0; the discipline is stated right in the text). M111 does the real work: a recursive descent over two expressions, testing connectives for identity, and when it meets a

free variable, *assigning* whatever stands opposite it as that variable's substitutor on a growing **substitution list**. Variables already assigned must match their previous assignment. If the recursion bottoms out everywhere, the expressions are unifiable, and the substitution list says how.

That list is "compact": its values are pointers into the actual expressions being matched, and may themselves contain variables needing further substitution. When a method needs to actually *build* something — a new subproblem with the substitutions applied — it calls M113, which matches and then expands the compact list via M112, recursively constructing fully-substituted local copies. The expansion subprocess is called, beautifully, **delineation**.

Before any match, M110 makes the free variables of problem and theorem **disjoint**, renaming the theorem's variables away from the problem's by drawing fresh ones from the system free-variable list L2.

★ DON'T MISS

L2 contains exactly seven free variables: A through G. M110's rename loop walks L2 looking for an unused letter, and if it runs out —

HALT DUE TO NOT ENOUGH FREE VAR.

70J7

M110R300

J7 is "Halt, proceed on GO." The machine stops dead and waits for the *operator* to push a button. No exception, no error message, no graceful degradation: the alphabet ran out, summon a human. Seven variables were enough for *Principia* Chapter 2, and not one letter more was provisioned.

What M111 implements is one-sided unification — substitution flows into free variables only, and bound variables act as opaque constants that can be substituted *for* free variables but never receive substitutions themselves. The footnote on bound variables ("treated as though they are segments of a particular but unknown form") is a 1963 sentence that any modern reader will recognize as describing Skolem-constant behavior. Full two-sided unification, with the occurs check and all, was Robinson 1965 — two years after this listing.

District 5: The Workshops — the other methods

None of the other methods proves anything. Each is a factory that converts the current problem plus one true expression into a new subproblem, runs it through finishing (M19) and customs (District 6), immediately tries substitution on it in the hope a proof is one step away, and

otherwise leaves it on the untried list.

- **M11 Detachment** uses modus ponens backward: match the whole problem against the *right* sides of theorems whose main connective is IMPLIES; a match makes the theorem's left side (suitably substituted) the new subproblem. Problem $\neg PVP$ and axiom $(AVB)I(BVA)$ yield subproblem $PV-P$.
- **M13 Replacement** matches the whole problem against either side of a *definition* (main connective $.=. .$) and rewrites it as the other side. Problem PIP and definition $(AIB).=. (\neg AVB)$ yield $\neg PVP$ — and vice versa.
- **M14/M15 Forward and backward chaining** implement transitivity of implication, both used backward: problem AIC plus theorem AIB yields subproblem BIC (forward); problem AIC plus theorem BIC yields AIB (backward). The text is scrupulous about the logic here: chaining isn't a *Principia* rule of inference, it's provable from *2.05/*2.06, and LT can prove everything up through *2.06 without using it — so the bootstrap is honest.
- **M16/M17 Sublevel replacement** — Stefferud's new methods — descend *inside* the problem, trying definitional rewrites on subsegments level by level: M16 from the top down, one new subproblem per level; M17 trying all sublevels into a single subproblem. This is what lets the 1963 LT crack theorems like *2.01 in one step where whole-expression replacement can't reach.

M19 finishes every newborn subproblem identically: attach the derivation attributes (from-problem, via-theorem, by-method), test utility, and either add it to the untried list or erase it.

A note on modern equivalents: the whole arrangement — a goal, a set of backward rule applications generating subgoals, immediate testing of each subgoal against known facts — is recognizably backward-chaining proof search, the skeleton later standardized in Prolog's SLD resolution. The difference is that LT's "resolution step" is the weaker one-sided match of District 4, and its search control is the heuristic open-list machinery of District 3 rather than depth-first with backtracking.

District 6: The Customs Office — utility measures

M43 decides whether a subproblem deserves to exist. The criteria are charmingly blunt. A problem has sufficient utility if it is not *clearly unprovable* — not a bare variable like P or $\neg P$, not structurally faulty, not of the form $X \vee X$ across a main OR (PVP , or $(PIQ) \vee (PIQ)$ — detected by matching the two sides against each other with M114) — and if it is *new*, meaning it doesn't already appear on the found-problems list L11.

The newness test M42 is where the engineering lives. Matching every new subproblem against

every previously found one would be ruinous, so L11 is structured as nested sublists indexed by three numbers: Q2 (number of levels), Q3 (number of distinct variables), Q4 (number of variable places). Only problems agreeing on all three are put through the full no-substitution match M40 — which treats any pair of free variables in corresponding positions as equal, so `PIP` and `QIQ` are the same problem. This is a duplicate-detection hash, with (depth, vars, places) as the hash key — the closed list of modern search, with three-field bucketing standing in for a hash function.

Rejected problems aren't silent: with K31 set to YES, they're printed and numbered, so the sample run's output is studded with lines like

```
5088      P      *2.02, DETACHMENT. REJECTED PROBLEM
```

— a number, a corpse, and the method that produced it. Reading the rejects is the fastest way to develop a feel for how much of LT's search is pruned at the border.

District 7: The Card Catalog — the axiom map

This district is Stefferud's own addition, developed (a footnote says) as his term project under Fred Tonge on the Western Data Processing Center's IBM 7090, and it solves what the text calls "the old problem of what to do with too much information."

The original LT screened candidate theorems by comparing global counts — levels, variables, variable places. This was expensive and, worse, *wrong*: the counts sometimes screened out the one theorem whose match was crucial, rendering problems unprovable. Stefferud's diagnosis is that an expression is characterized by its *form*, and the essence of form is connective structure. So all true expressions are mapped onto one tree, **L4, the Map of All True Expressions**: a description list with connectives as attributes and submaps as values, where each submap position corresponds to a variable-or-segment place, and each submap's head lists every theorem that has a *variable* at that place.

A theorem is a feasible match for a problem if it has the same connective structure, "viewing the problem as contracted" — i.e., treating any problem segment as collapsible to a variable wherever the theorem has a variable, which corresponds exactly to what the matcher's substitution can absorb. Retrieval (M63, with recursive worker M62) lays the problem over the map, collects theorem names from the overlaid heads, and intersects across submaps. The worked example in the text: for problem `(PVQ)I(PV(PVQ))`, axioms *1.2, *1.3, *1.4 are feasible; *1.5 is eliminated by the intersection (its name is missing from one required head), and *1.6 *is never even visited* — the walk never descends into map regions with the wrong

connectives.

Each method consults only its own wing of the catalog: M12 the whole map, M11 only the submap of right-sides-of-IMPLIES, the replacement methods only the `.=.` submaps.

"Obviously irrelevant true expressions no longer get in the way," the text concludes, and notes that the technique's value *grows* with the size of the theorem store.

★ DON'T MISS

What you are looking at is term indexing — a discrimination tree over connective skeletons, with retrieval-by-overlay and per-rule index restriction — the family of techniques that modern automated theorem provers and Prolog implementations reinvented and industrialized from the 1980s onward. Stefferud even sketches the next steps as "student exercises": index the found-problems list the same way (noting, correctly, that its no-substitution match needs the *opposite* viewing-as-contracted convention), and search the theorem map and an untried-problem map simultaneously so the executive could "see" more of its problem at once. Sixty years later those exercises read like a research program that actually happened — minus the citation.

District 8: The Machine Shops — the P and Q routines

The lower-level routines operate on expression structures and terms, segregated, the text says, so that the representation could in theory be changed by replacing only this layer. Expressions are binary trees of cells: a segment is a list whose head holds the connective and whose two elements are subsegments or variable terms; a *total expression* wraps the main segment with a description list carrying attributes like Q7 (external name, e.g. `*1.5`) and Q15 (tree form).

The **Q routines** are the district's showpiece. Q1–Q19 are "find" routines for attributes, and several are what the text calls **active attributes**:

```
Begin Q2 [Find number of levels of total expression (0)]
  Find number of levels on description list (J10)
    if found → quit + (the cached answer)
    if not   → count levels recursively (9-100),
               assign count as value of Q2 on the
               description list, output it
```

Compute once, cache on the object, serve from cache thereafter. Q3 (distinct variables) and Q4

(variable places) work the same way. The customs office's three-field index keys (District 6) are these cached values — memoization in the service of hashing, in 1963.

★ DON'T MISS

Q7 is simultaneously (a) the name of the routine FIND EXTERNAL NAME, (b) the attribute EXTERNAL NAME on description lists, and (c) *its own value* in places where only the attribute's presence matters. The text explains the pun with a straight face: since description-list processes never operate on the contents of cells used as attributes, "the attribute symbol may also serve as its own dummy value, as well as for the name of the routine used to find it." One symbol, three jobs, zero conflicts — name punning as a memory-conservation strategy on a machine with 17,978 free cells after loading (the loader prints the number).

The **P routines** include the generators (P26 generates segment locations at a given level, using a subgenerator that runs *another* generator as its subprocess — iterators composed with iterators) and the input conversion chain P50–P52, which turns the parenthesized "list form" punched on cards into internal trees, splicing `/UGH/` (District 1) wherever the recursion fails. P27 replaces bound variables by free ones — remember that routine; it's the hinge of the final district's best trick.

District 9: The Press Office — M70–M89

M89 reads one logic expression per card: locate the name, check it's a regional symbol, build a list-form expression from the characters, attach any suffix (like `DEF.`) as attribute Q18. M73 is the reverse: a recursive tree-to-characters walk that prints expressions into the line buffer, entering `/UGH/`'s external name and *continuing* if it finds the tree faulty.

M71 prints whole proofs by walking the derivation attributes (Q10–Q14) backward from the successful subproblem — the proof was never stored as a proof; it's reconstructed from the tree's breadcrumbs, exactly like a modern search algorithm recovering a path from parent pointers:

```
PROOF FOUND.
  GIVEN                (AVA) IA
  SUBSTITUTION          (-PV-P) I-P
  GIVEN                DEFINITIONS
  SUBLEVEL REPLACEMENT *2.01 (PI-P) I-P
  Q.E.D.
```

★ DON'T MISS

That line `GIVEN DEFINITIONS`. The proof printer prints *expressions*, by recursive descent — left subtree's name, connective, right subtree's name. The sublevel replacement methods don't use a single theorem, so what should the "theorem used" slot say? The data section answers with `/16`, the "dummy expression": a fake total expression whose left operand is a dummy variable with external name `DEFIN`, whose right operand is a dummy variable with external name `TIONS`, and whose main connective is `I` — IMPLIES, which prints as the letter I. So the expression printer renders `DEFIN ⊃ TIONS` as

```
DEFINITIONS
```

The word "DEFINITIONS" in every sublevel-replacement proof you'll ever see from LT is a well-formed logic expression asserting that `DEFIN` implies `TIONS`. It may be the single best hack in the listing: rather than teach the printer to print words, Stefferud taught a word to be an expression.

One more press-office ritual closes each successful proof. If constant K30 holds `R`, M2 prints `REMEMBER PROVED THEOREM`, runs the theorem through P27 — bound variables become free — and M50 adds it to the true-expressions list *and* the axiom map. Prove `PIP` and the catalog gains `AIA`. The bound-to-free promotion is the type-level ceremony of theoremhood: the conjecture's rigid P becomes a substitutable A, available to every later proof. LT, in this precise and bookkept sense, learns — and the sample run shows later proofs leaning on remembered ones (*2.06's proof uses freshly-remembered *2.05 and *2.04).

The Mayor's Office and the Bureau of Debugging

X1 is the run executive: initialize the debugging traps, read the true expressions (converting each and adding it to the map; failures are printed via M88 and dropped — exit *2.33), read the problems onto L3, and hand control to M2. X9 builds a restart tape of the whole loaded system so class sessions didn't have to re-load the deck.

The Bureau (X10–X19, with lists X21–X23) is a complete 1963 debugging kit: X10 flips on full trace when the cycle counter H3 hits a preset value W33 — "trace only after considerable running time" — and X11 revokes it; X13 snapshots H0 at routine entry and exit; X14 cuts a restart tape on an operator's console signal; X15 extends the post-mortem by printing the entire axiom map. Effort limits are enforced by M90 against K20 (subproblems), K21 (substitutions), and K22 (effort) — and "effort" is nothing abstract: M3 snapshots H3, the interpreter's own cycle

counter, as the effort base, so LT's resource bound is literally machine cycles. When the run ends, the standard IPL-V post-mortem dumps every system cell — the 1963 sample run signs off with H1 holding J7 and H5 holding J4: halted, last test positive.

★ DON'T MISS

The entire body of X19, MONITOR POINT FORCER:

```
x19    03J0    0
```

One instruction: execute J0 — the no-op — with Q=3, the addressing mode that *starts selective trace*. A routine whose only content is doing nothing, traceably, so that pending trace-mode changes take effect immediately. It is hard to imagine a smaller useful routine, and the listing's comment field labels it without irony.

A Day in the Life: One Theorem, End to End

The run deck is loaded; the axioms and definitions are read, converted by P50, and mapped into L4 (*2.33 dies at the border with an /UGH/). The first problem is ***2.01**, **(PI-P) I-P**.

1. **M2** pops *2.01 off L3, converts it to tree form, sees K30 = **R**, and calls **M1** in remembering mode.
2. **M43** finds the problem useful (not a bare variable, not $X \vee X$, not previously found; its Q2/Q3/Q4 get computed and cached on the way). M78 prints
TO PROVE *2.01 (PI-P) I-P.
3. **M12** consults the map for feasible whole-expression matches and finds nothing that unifies — no axiom has this shape. H5-.
4. **M8** builds the method list. *2.01 is an *original* problem, so the special list L6 goes in front: **M16** runs first.
5. **M16** descends to the sublevels of the problem, asks the map's **.=.** wing for definitions whose side matches the subsegment **(PI-P)**, and finds *1.01, **(AIB) .=. (-AVB)**. M110 makes variables disjoint, M113 matches and expands the substitution list, the segment is rewritten, and M19 finishes the new subproblem **(-PV-P) I-P** — derivation attributes set, with **/16** (that is, DEFINITIONS) in the theorem slot. Customs passes it.
6. Per the standing rule, the newborn goes straight to **M12** — and this time the map offers *1.2, **(AVA) IA**. M111's recursion assigns substitutor **-P** to free variable **A**, everywhere consistently. Match. Proof.
7. **M71** walks the derivation attributes backward and prints the proof shown in District 9, then the effort report: in the 1963 run,

EFFORT LIMIT 200000 ACTUAL 8062, SUBPROBLEMS 1, SUBSTITUTIONS 2 .

8. **M82/P27/M50:** REMEMBER PROVED THEOREM (AI-A) I-A — bound P promoted to free A — and *2.01, now an axiom in all but name, is threaded into the map for every theorem that follows.

In the Memorandum's full sample run, with the effort limit at 200,000 cycles, LT proves 24 of the 25 theorems attempted; only *2.15 fails, dying at effort 206,728 with 31 subproblems explored. The 2026 reanimated run, with a tighter 20,000-cycle limit, proves 16 of 23 — and watching *which* theorems fall out as the budget shrinks is exactly the kind of experiment the reanimation makes possible.

Practical Notes for the Independent Traveler

On reading the card format. Each line of the listing is one card: comment field on the left; then NAME (blank except where a routine, label, or data item is being defined); then PQ and SYMB (the instruction); then LINK (the next instruction or local label — blank means "fall through," 0 means "end of routine"); and on the far right the card identifier (M001R060 = routine M1, card 60), the line numbers of the punched-card era. Memorize four readings and the listing opens up: 00X = call X; 10X = push the symbol X onto H0; 11X = push the *contents* of cell X; 70... = branch if the last test failed. 40W0 / 30W0 bracketing a routine is push/pop of working storage — IPL-V's calling convention, written out longhand every time.

On the two notations. The Memorandum's prose uses square brackets — [AVA]IA — while the deck and the output use parentheses. Same expressions; the brackets are just for human eyes. Output from the 1963 line printer (reproduced photographically in the report) renders parentheses as I-like marks that OCR mangles enthusiastically; when a sample-run line looks like IAVAiIA , you are looking at (AVA)IA .

On the structure of the document. RM-3731 is itself a guided tour: Sections II–XI are the prose walk, Section XII a complete sample run (input deck and all output), Section XIII the symbol-naming conventions, XIV the vocabulary (every symbol, one line each — the single most useful page to keep open while reading code), and XV the full listing. The vocabulary is deliberately maintained as an extension of the IPL-V manual's List of Basic Processes; with both in hand, every symbol in the listing is resolvable.

On what's missing or repaired. The listing in the report is complete, but it's a *photograph of a printout*, and transcription was genuinely hard: the code was transcribed to a spreadsheet by Shrager and Hay, with residual errors caught later by Rupert Lane's "gridlock" PDF-extraction tooling — and some survived multiple rounds of human review anyway. The runnable file

`LTFixed.liplv` in the reanimation repo differs from the paper in a small number of documented ways (BCD character-set workarounds, parameters moved to the end for convenience); more than 99% is the 1963 code verbatim. One deeper repair came not from any document but from an artifact: the interpreter's J74 (copy list structure) matched the manual's prose yet broke LT, and the discrepancy was resolved only against "Simon's J's," the original 1962 J-function punched cards preserved by the Computer History Museum. The manual underdetermined the machine; only the cards knew.

On running it. Clone github.com/jeffshrager/IPL-V, install SBCL, and

`sbcl --eval '(load (compile-file "lt.lisp"))'`. The full 23-theorem run takes about 574,000 IPL cycles — seconds on a laptop, plausibly hours on the JOHNNIAC — and emits timestamped logs plus Graphviz proof-search graphs (green for methods that succeeded, pink for failures, gold and gray for subproblems accepted and rejected): the subproblem tree of Figure 2, rendered for real, for every theorem.

What Makes the Logic Theorist Worth Visiting

The history books tell you LT was the first AI; the listing tells you what that meant in cells and cards. Strip away the legend and you find a clean, modular search program: an open list and a closed list, backward-chaining subgoal generation, one-sided unification with substitution lists, duplicate detection bucketed by cheap memoized features, term-indexed retrieval, resource bounds measured in machine cycles, and a learning loop that promotes proved conjectures into the index. Nearly every load-bearing idea of symbolic AI's first three decades is present, in miniature, with comments.

Two things about the visit linger. The first is the gap this version makes visible: the 1956 LT was ~400 lines of hand-executed pseudocode — close to a bare statement of Newell and Simon's theory of human theorem proving — while this fully mechanical 1963 descendant runs to nearly 3,000. The difference is the part the "human computer" used to do silently (the 1956 listing, it turns out, couldn't even *print its proofs* — Simon's students just wrote down what they had done). Most of AI, then as now, lives in that gap.

The second is the document's intent. Stefferud wasn't preserving a monument; he was building a teaching machine, and he ends sections with open problems — better utility measures, maps over the untried problems, executives that plan. "They are not the only or the best ways," he writes of his own design choices, "but they do enable LT to do a passable job of theorem proving." It's an invitation, and after sixty years of automated reasoning built on exactly these foundations, it still reads like one.

Provenance and Sources

Claims about LT5's code and behavior come from the complete program listing, vocabulary, and sample run in Stefferud (1963), RM-3731, cross-checked against the transcription

`LT/LT.iplv` in the IPL-V repository; these are high confidence where the code is explicit (the /UGH/ marker, the M110 halt, the /16 DEFIN-I-TIONS structure, the Q-routine caching, X19's body, the L6/L7 method-list split, the 1963 sample run's 24-of-25 result with only *2.15 failing at the 200,000-cycle limit). The OCR of the report is imperfect, so exact characters in quoted *output* lines are normalized from garbled glyphs (moderate confidence on exact rendering; high on content). The historical narrative — LT1/LT2 lineage, JOHNNIAC dates, the 38-of-52 result, the Russell anecdote, the transcription and debugging history, the J74/"Simon's J's" episode, and the reanimation's 16-of-23 result at a 20,000-cycle limit — follows Moews & Shrager (2026), arXiv:2603.13514, and the repository README (high confidence as to what those sources claim; the underlying 1950s anecdotes are as reliable as McCorduck's interviews, i.e., standard but secondhand). Reanimation result tables in the README and the paper differ on a few theorems, consistent with runs under different limits; the paper's table is treated as canonical here (moderate confidence). Characterizations of the axiom map as anticipating discrimination-tree term indexing, and of L10/L11 as open/closed lists, are the author's interpretive framing of mechanisms that are themselves explicitly documented (the mechanisms: high confidence; the framing: interpretation).

////////////////////////////////////

References

Works consulted in preparing this guide; not all are cited above.

McCorduck, P. (2004). *Machines Who Think* (2nd ed.). A. K. Peters.

Moews, D., & Shrager, J. (2026). "Executable Archaeology: Reanimating the Logic Theorist from its IPL-V Source." arXiv:2603.13514.

Newell, A., & Simon, H. A. (1956). *The Logic Theory Machine: A Complex Information Processing System*. RAND P-868; also *IRE Transactions on Information Theory*, IT-2(3), 61–79.

Newell, A., Shaw, J. C., & Simon, H. A. (1957). "Empirical Explorations of the Logic Theory Machine: A Case Study in Heuristic." *Proceedings of the Western Joint Computer Conference*, 218–230.

Newell, A. (Ed.). (1964). *Information Processing Language-V Manual* (2nd ed.). Prentice-Hall.

Shrager, J. *IPL-V repository* (interpreter, LT transcription, run outputs).
github.com/jeffshrager/IPL-V

Moews, D. *logic-theorist repository* (IPL-I interpreter and repaired LT1 source).
github.com/dmoews/logic-theorist

Simon, H. A. (1996). *Models of My Life*. MIT Press.

Stefferd, E. (1963). *The Logic Theory Machine: A Model Heuristic Program*. RAND
Memorandum RM-3731-CC.

Whitehead, A. N., & Russell, B. (1910). *Principia Mathematica*, Vol. 1. Cambridge University
Press.

Bochanek, A. (2012). "Simon's J's." Computer History Museum blog.

Lane, R., Hay, A., Schwarz, A., Berry, D. M., & Shrager, J. (2025). "ELIZA Reanimated."
[arXiv:2501.06707](https://arxiv.org/abs/2501.06707).

Copyright

Hacker's Guides — guide texts, commentary, and supporting material.

Copyright © 2026 Jeff Shrager (jshrager@gmail.com). All rights reserved.

Permission requests, corrections, and questions: jshrager@gmail.com.

Scope

This copyright covers the original text of the guides and the README — the prose, structure, commentary, and annotations written for this repository:

<https://github.com/jeffshrager/HackersGuides>

It does **not** cover:

- **The historic source code being read.** Quoted program source (and any complete source files included in this repository) remains the property of its respective authors and rights holders, or is in the public domain, as the case may be. Excerpts appear here for purposes of commentary, criticism, and scholarship. Provenance for each program's source is stated in the corresponding guide.
- **Quoted third-party material.** Brief quotations from cited historical accounts and

documentation remain the property of their respective rights holders and are used for the same purposes.

Attribution

If you quote or reference a guide, please credit "Hacker's Guides, Jeff Shrager" and link to this repository: <https://github.com/jeffshrager/HackersGuides>